

Práctica 1: Sistemas difusos con FuzzyCLIPS

Estabilización del vuelo de un dron

Técnicas, Entornos y Aplicaciones de Inteligencia Artificial (TIA)

Curso 2026–2027



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



1. Conceptos básicos de lógica difusa en FuzzyCLIPS

- Variables, valores y funciones de pertenencia
- Modificadores lingüísticos y expresiones lógicas
- Hechos difusos y fusificación
- Reglas, inferencia y defusificación
- Hechos estructurados, prioridad (**salience**) y control de la ejecución

2. El problema: estabilización de un dron

- Entradas, salidas y estrategia de control
- Diseño pedido y funcionamiento
- Resultados de referencia para validar

Referencia completa: boletín de la práctica y manual de FuzzyCLIPS (PoliformaT).

Conceptos básicos

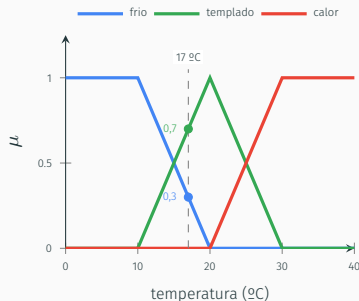
¿Por qué lógica difusa?

Lógica clásica (crisp)

- Se pertenece o **no** a un conjunto.
- 19,9 °C es “frío”; 20,1 °C ya no.

Lógica difusa (fuzzy)

- Se pertenece con un **grado** $\mu \in [0, 1]$.
- 17 °C es “frío” (0,3) y “templado” (0,7) a la vez.
- Razonamos con términos imprecisos:
alto, poco, muy frío...



FuzzyCLIPS = CLIPS + hechos y reglas difusas: mezcla información crisp y difusa en la misma base de conocimiento.

Una variable difusa se declara con un **deftemplate** que define:

nombre · **universo** (mín., máx., unidades) · **valores lingüísticos** · **función de pertenencia** de cada valor

```
(deftemplate temperatura           ; variable difusa
  0 40 grados                      ; universo y unidades
  ( (frio      (0 1) (10 1) (20 0)) ; puntos (x  $\mu$ )
    (templado (10 0) (20 1) (30 0))
    (calor    (20 0) (30 1)) ))
```

- Cada valor se describe con **puntos característicos** ($x \mu$), unidos por rectas.
- Antes del primer punto y después del último, μ se mantiene constante hasta los límites del universo.
- Es la variable de la gráfica anterior.

Formas de definir funciones de pertenencia

Por puntos (cualquier forma)

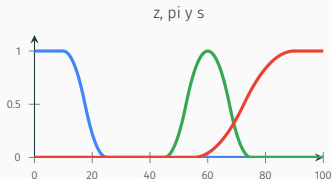
```
(medio (100 0) (150 1)  
      (170 1) (180 0))
```

Funciones predefinidas

```
(bajo (z 10 25)) ; Z  
(medio (pi 15 60)) ; campana  
(alto (s 55 90)) ; S
```

Singleton (un único valor exacto)

```
(tres (3 0) (3 1) (3 0))
```

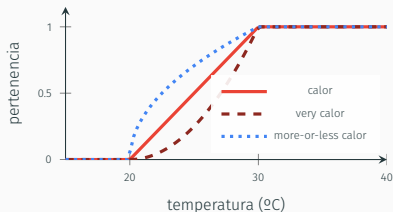


(pi d b): campana centrada en b con semiancho d. Más detalle en el manual.

Modificadores lingüísticos

Transforman un conjunto difuso ya definido:

Modif.	Efecto
not	$1 - \mu$
very	μ^2
extremely	μ^3
plus	$\mu^{1.25}$
somewhat	$\mu^{1/3}$
more-or-less	$\sqrt{\mu}$



Sirven para **definir nuevos valores**, combinados con **AND**, **OR**, **NOT**:

(mucho-calor **very** calor)

(templado **not** [frio OR calor])

¡Espacio después de [y antes de]!

Un hecho difuso asocia una variable difusa a un conjunto difuso:

```
(temperatura calor)                ; valor definido
(temperatura very calor)           ; con modificador
(temperatura frio OR templado)     ; expresión lógica
(temperatura (15 0) (18 1) (21 0)) ; conjunto nuevo "al vuelo"
```

- **AND** = mínimo, **OR** = máximo, **NOT** = $1 - \mu$. **AND** tiene más prioridad que **OR**.
- Se introducen con (**assert ...**) o como hechos iniciales en (**deffacts ...**).
- Dos **assert** sobre la misma variable **no** equivalen a **AND**: FuzzyCLIPS los combina con **OR** (unión).

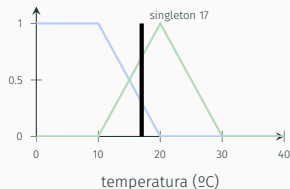
Cuidado

Un hecho **no estructurado** que *mezcla* varias cosas, como (**juan edad adulta**), se trata como **crisp**: no hará matching difuso.

Fusificación de valores crisp

Los sensores dan **números** (valores crisp), pero las reglas trabajan con **conjuntos difusos**. Hay que *fusificar*.

- Función **fuzzify** (apartado 2.3 del boletín; hay que copiarla al fichero).
- Aserta un triángulo centrado en el valor con semiancho **delta**.
- Con **delta = 0** $\Rightarrow$ singleton.



```
(fuzzify temperatura 17 0)
;; equivale a (temperatura (17 0) (17 1) (17 0))
```

Por consola: (**read**) devuelve el número y se pasa a **fuzzify**. Un valor difuso leído no puede asertarse directamente (habría que usar **assert-string**).

Ojo: valores crisp justo en los extremos del universo pueden dar problemas.

```
(defrule hace-calor
  (temperatura calor)           ; antecedente difuso
  =>
  (assert (ventilador rapido))) ; consecuente difuso
```

- **Antecedente:** patrones sobre hechos difusos (con modificadores y expresiones lógicas), hechos crisp y **test** sobre valores crisp.
- La regla se dispara en la **medida** en que el hecho encaja con el patrón (grado de matching). Varios patrones difusos se combinan con **mínimo**.
- **Consecuente:** **assert**, **printout**, **modify**... como en CLIPS.
- Capturar el valor difuso: **(temperatura ?t)** o **?h <- (temperatura ?)**.

Limitaciones

- **test** no hace matching difuso: **(test (eq ?t calor))** no sirve.
- Para que un número encaje con un valor difuso hay que fusificarlo antes.

Inferencia difusa

Entrada **17 °C**, dos reglas se activan:

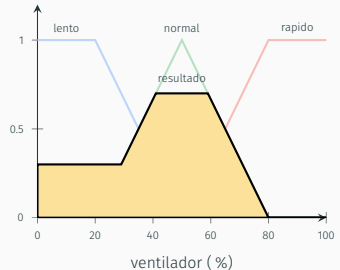
- **frio** $\Rightarrow$ ventilador lento
(grado **0,3**)
- **templado** $\Rightarrow$ ventilador normal
(grado **0,7**)

Max-min (por defecto): cada consecuente se *recorta* a su grado y se unen (OR).

Max-prod: se *escala* en vez de recortar.

(set-fuzzy-inference-type max-prod)

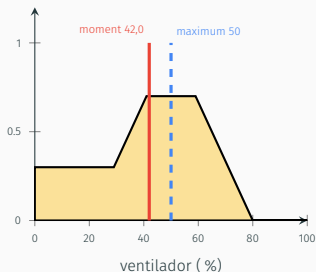
Varias reglas que asertan la misma variable se **agregan por unión** en un único hecho difuso (y las reglas que dependen de él pueden volver a dispararse).



Convierte el conjunto difuso resultante en **un número** para el actuador.

- **moment-defuzzify**: centro de gravedad (usa toda la forma).
- **maximum-defuzzify**: media de los máximos (solo la zona más alta).

```
(defrule defusificar
?v <- (ventilador ?)
=>
(printout t
(moment-defuzzify ?v)  crlf
(maximum-defuzzify ?v)  crlf))
```



Los dos métodos pueden dar valores **muy distintos**: comparar ambos es parte del análisis de la práctica.

Hechos estructurados (templates con slots)

Además de variables difusas, un **deftemplate** puede definir un **hecho estructurado** con *slots* crisp:

```
(deftemplate sala
  (slot nombre (type SYMBOL))
  (slot temp   (type FLOAT) (default 0.0)))
```

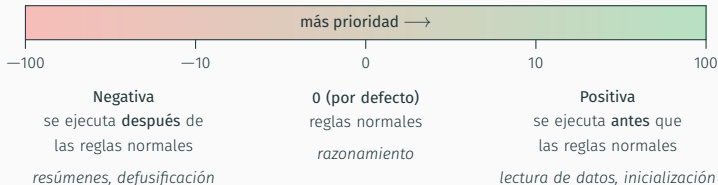
```
(def facts inicio (sala (nombre aula1)))
```

```
(defrule actualizar
  ?s <- (sala (nombre aula1) (temp 0.0))
  =>
  (modify ?s (temp 21.5)))
```

- Para cambiar slots usa **modify** sobre la instancia, **no** un nuevo **assert** (crearía otra instancia).
- Si cambian varios slots, un **único modify** con todos ellos.
- En una regla se pueden mezclar patrones sobre slots crisp y sobre variables difusas.

Prioridad de reglas: **salience**

Cuando hay **varias reglas activadas** a la vez, se ejecuta primero la de mayor **salience**. Es un entero; si no se declara vale 0.



```
(defrule inicializar (declare (salience 10)) ...) ; primero
(defrule normal ...) ; salience 0
(defrule resumen (declare (salience -10)) ...) ; al final
```

Ojo

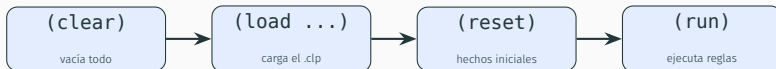
salience solo **ordena** las reglas ya activadas: no hace que una regla se dispare si sus condiciones no se cumplen.

Detener la ejecución

```
(defrule ultima
  (declare (salience -100))
  ...
=>
  ...
  (halt))           ; detiene la inferencia
```

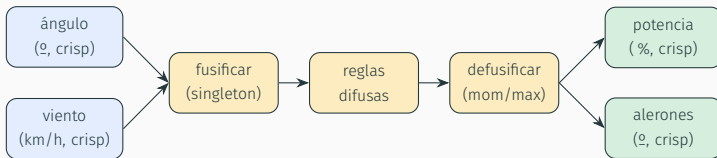
- **(halt)** termina la ejecución aunque queden reglas pendientes: evita bucles por el refinamiento de hechos difusos.
- Combinado con una **salience** muy negativa, garantiza que esa regla sea la **última** y se ejecute **una sola vez**.

Ciclo de trabajo en el entorno



El problema: estabilización de un
dron

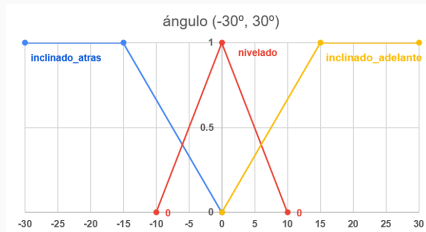
Un dron debe mantenerse **equilibrado** ante ráfagas de viento imprevistas.
Diseñaremos un controlador difuso que, en tiempo real:



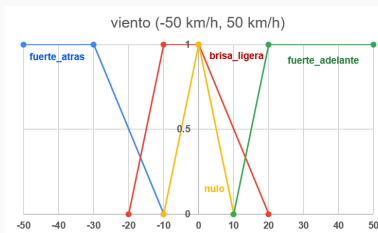
- **Entradas:** sensor de inclinación (ángulo sobre la horizontal) y sensor de viento (velocidad).
- **Salidas:** potencia de los motores y posición de los alerones.
- Conocimiento de los expertos: una **tabla de control** + tres **pruebas de telemetría**.

Variables de entrada

ángulo (sensor de inclinación) · universo $[-30, 30]$ grados

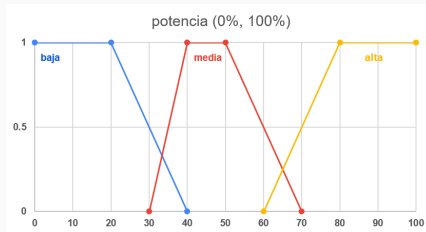


viento (sensor de viento) · universo $[-50, 50]$ km/h

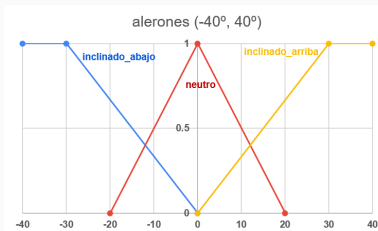


Variables de salida

potencia (motores) · universo $[0, 100] \%$



alerones (posición) · universo $[-40, 40]$ grados



Consejo: leer bien las gráficas

Antes de escribir nada, pasa cada gráfica a **puntos característicos** (x μ).

- Fíjate en qué puntos tienen $\mu = 1$ y cuáles $\mu = 0$ para cada valor.
- Hay funciones **trapezoidales** (con meseta) y **triangulares**.
- Los valores extremos (p. ej. **inclinado_atras**, **fuerte_adelante**) se mantienen a 1 hasta el límite del universo.
- Observa los **solapamientos**: un mismo valor crisp puede pertenecer a dos conjuntos a la vez.

Comprueba tus funciones

Dibuja los conjuntos en la consola y compáralos con la gráfica original:

```
(plot-fuzzy-value t "*+." nil nil
  (create-fuzzy-value angulo inclinado_atras)
  (create-fuzzy-value angulo nivelado)
  (create-fuzzy-value angulo inclinado_adelante))
```

Estrategia de control: [potencia, alerones]

ángulo / viento	fuerte_atras	brisa_ligera	nulo	fuerte_adelante
inclinado_atras	alta, inclinado_abajo	media, somewhat inclinado_abajo	baja, inclinado_arriba	more-or-less baja, neutro
nivelado	media, inclinado_abajo	somewhat media, neutro	media, somewhat neutro	alta, very inclinado_arriba
inclinado_adelante	very baja, neutro	baja, neutro	alta, very inclinado_abajo	extremely alta, inclinado_arriba

- Cada celda: *si ángulo es X y viento es Y entonces potencia es ... y alerones es ...*
- Los modificadores (*very, somewhat...*) afectan al valor de salida que se aserta.
- Todas las celdas siguen el mismo esquema: diseña **una** regla, compruébala y replica.

Tres casos medidos en vuelo que **también hay que modelar como reglas**:

Prueba	Inclinación (°)	Viento (km/h)	[potencia, alerones]
P1	-5	-30	media, neutro
P2	20	35	alta, inclinado_arriba
P3	-15	-40	alta, inclinado_abajo

- Vienen dadas con valores **crisp**, pero las reglas se escriben con **valores difusos**: mira en las gráficas a qué conjuntos pertenece cada valor.
- Ejemplo del boletín (P1):
 - inclinación -5 está en dos conjuntos
⇒ (**angulo inclinado_atras** AND **nivelado**)
 - viento -30 está solo en uno ⇒ (**viento fuerte_atras**)
- Aplica el mismo razonamiento a P2 y P3.

El template Dron

Además de las 4 variables difusas (ángulo, viento, potencia, alerones), hay que definir un template **Dron** con una **única instancia** que guarde entradas y salidas:

	Slot	Contenido
Inputs	sensor-inclinacion-crisp	ángulo medido (grados)
	sensor-viento-crisp	velocidad del viento (km/h)
Outputs	mom-potencia-crisp	potencia (%) por <i>moment</i>
	max-potencia-crisp	potencia (%) por <i>maximum</i>
	mom-alerones-crisp	alerones (grados) por <i>moment</i>
	max-alerones-crisp	alerones (grados) por <i>maximum</i>

- Todos los valores crisp de tipo **FLOAT**.
- Nombres **sin tildes**.

Funcionamiento esperado

1. **Entrada.** Pedir al usuario la inclinación y el viento (valores crisp) y guardarlos en la instancia de **Dron**.
2. **Fusificación.** Convertir ambos valores con **fuzzify** y **delta = 0** (singleton).
3. **Razonamiento.** Las reglas de la tabla y de la telemetría infieren **potencia** y **alerones**.
4. **Salida.** Defusificar ambas variables con **moment-defuzzify** y **maximum-defuzzify**, guardar los 4 resultados en la instancia y mostrarlos por pantalla.

Requisitos de la regla de defusificación

- Debe ser la **última** en ejecutarse (**salience**) y hacerlo **una sola vez**.
- Termina con (**halt**) para evitar bucles.
- Actualiza la instancia con **un único modify**.

Resultados de referencia (fusificación singleton)

Entradas		Potencia (%)		Alerones (°)	
incl.	viento	mom	max	mom	max
-5	-30	62.81	47.5	-14.48	-27.5
-5	8	41.08	48.73	-3.39	0.0
-5	30	55.52	85.0	14.40	30.6
20	-30	13.77	10.0	$\approx 0^*$	0.0
20	8	40.36	47.0	-10.80	-25.42
20	30	84.44	90.0	26.0	35.0

Valores truncados a dos decimales. * FuzzyCLIPS muestra $-1.42\text{E-}15$ (error de redondeo).

- Úsalos para **validar** tu implementación antes de probar otros escenarios.
- Después: evalúa escenarios nuevos y **compara** *moment* vs *maximum*.
¿Cuándo difieren más? ¿Por qué?

Consejos y errores frecuentes

- **Desarrollo incremental:** variables → fusificación → *una* regla → defusificación → resto de reglas.
- Usa la ventana *Facts* para ver los conjuntos difusos que se van generando.
- Siempre (**clear**), cargar, (**reset**), (**run**). Sin (**reset**) no hay hechos iniciales ni instancia.
- **fuzzify** hay que **copiarla** al fichero; con **delta = 0**.
- **No** asertar una nueva instancia de **Dron**: usar **modify**.
- La defusificación solo debe ocurrir **una vez**, al final: revisa **salience** y (**halt**).
- **test** no sirve para comparar valores difusos.
- Si un resultado no coincide, revisa primero los **puntos** de las funciones de pertenencia y los **modificadores** de la tabla.

La evaluación consistirá en **rediseñar y evaluar pequeñas modificaciones** del problema: entiende bien tu solución.

Ampliación voluntaria: botón ECO

No necesaria para la evaluación.

- Nueva entrada: botón **ECO (ON / OFF)**, que también se pide al usuario.
- Si está en **ON**, se aserta además **potencia baja** para ahorrar energía.
- Recuerda: un nuevo **assert** sobre una variable difusa se **une (OR)** con lo que ya hay.

Resultados de referencia con ECO = ON					
incl.	viento	mom-pot.	max-pot.	mom-aler.	max-aler.
-5	8	28.32	10.0	-3.39	0.0
-5	30	40.12	10.0	14.40	30.6
20	8	28.78	10.0	-6.01	0.0
20	30	50.0	50.0	26.0	35.0

Compara con la tabla sin ECO: ¿qué salidas cambian, cuánto y por qué?

Tarea y Evaluación

Tarea:

- Realizar el ejercicio de forma individual.
- La evaluación de las prácticas presupone que se hayan realizado las tareas asociadas a estas y obtenido el código correspondiente al problema planteado en cada práctica.

Calendario

- **28 de septiembre y 5 de octubre:** laboratorio de FuzzyCLIPS.
- **3 de noviembre:** evaluación de la práctica 1 de FuzzyCLIPS.